

5/07/20

2) Define a structure in 'c' as well as 'cpp'. This structure hold the variable for student type. compute the size manually for the structure and also use `sizeof` function to display the size. cross check if both are same or not. Justify your answer.

Eq:- Declare self referential structure for the following :- (i) emp
(ii) student
(iii) node.

```
(i) struct emp {  
    int eid;  
    char name[50];  
    struct emp * next;  
};
```

```
(ii) struct node {  
    int data;  
    int data;  
    float data;  
    struct node * ptr;  
};
```

```

(11) struct student {
    int rno;
    char name[50];
    struct student *next;
};

```

H.W using C

```

1) #include <stdio.h>
   struct student {
       int roll;
       char grade;
       float marks;
   };

```

output:-

Manual size = 9 bytes
 size using sizeof() =
 12 bytes
 Both are not same.

```

   int main() {
       int manual = 4 + 1 + 4;
       printf("Manual size = %.d bytes\n", manual);
       printf("size using sizeof() = %.d bytes\n",
           sizeof(struct student));
       if (manual == sizeof(struct student))
           printf("Both are same");
       else
           printf("Both are not same");
   }

```

using in C++

```

# include <iostream>
using namespace std;
struct students {
    int roll;
    char grade;
};

```

```

float marks;
};
int main () {
    int manual = 4 + 4 + 4;

    cout << "manual size = " << manual << " bytes" << endl;
    cout << "size using sizeof () = " << sizeof (struct student) << " bytes" << endl;
    if (manual == sizeof (struct student))
        cout << "Both are same";
    else
        cout << "They are not same";
    return 0;
}

```

The manual size and the size returned by `sizeof ()` is not the same usually because of "structure padding" which is known as "memory alignment".

generally,

int = 4 bytes
 char = 1 byte
 float = 4 bytes

however,

the compiler stores like int = 4 bytes, char = 1 byte, padding = 3 bytes, float = 4 bytes.

so they are not same because compiler adds padding bytes to align the data member in memory. It improves the performance of memory access.

Recursion:-

Q) sum of "n" numbers:

```
#include <stdio.h>
```

```
void main()
```

```
{
```

```
printf (sum(n)); }
```

```
if (n == 1)
```

```
return 1;
```

```
else
```

```
return n + sum (n-1);
```

09/07/21